

Testing pure components

Project: Grounded Learning Service

1. Your mission

Your learning target

I can explain the mechanism and terminology of testing pure components.

I can apply the stated procedure to a new case with a traceable result.

I can identify an assumption or limitation and design a revealing follow-up.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Least privilege and tool scopes. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Configuration and secrets. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Requirements and service boundaries. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Explain the mechanism and terminology of testing pure components

Apply the stated procedure to a new case with a traceable result

Identify an assumption or limitation and design a revealing follow-up

Retrieve what you already know

Recall Least privilege and tool scopes. Explain why a recorded result needs its input conditions and evidence source before it can support a claim.

My starting explanation

This week's end goal

Explain testing pure components using the deciding step in this fictional service.

2. Notice the evidence

Pure components return results from supplied inputs without external side effects. They are easier to test with exact expected values.

Begin by writing the question the procedure is intended to answer. Identify the information already supplied, the information it will produce and the information still missing. These are constructed classroom examples; they do not require live accounts, private records or an external service. Do not treat a plausible explanation as a measured outcome.

List the given facts and one fact that is not supplied.

3. See how it works

Define `remaining(capacity, used)=capacity-used` for integer inputs satisfying $0 \leq \text{used} \leq \text{capacity}$. Invalid types/ranges raise an explicit validation error. Test zero and equality boundaries.

Testing pure components — evidence trace

Input and scope · Check or calculation · Conclusion and limit

Input and scope: Capacity 12, used 5; capacity 12, used 12; capacity 12, used 13.

Check or calculation: Define `remaining(capacity, used)=capacity-used` for integer inputs satisfying $0 \leq \text{used} \leq \text{capacity}$. Invalid types/ranges raise an explicit validation error. Test zero and equality boundaries.

Conclusion and limit: Results 7 and 0; the third is invalid rather than remaining -1. Preserve the contract failure.

The figure connects this chapter's worked input, declared procedure and result. Read the complete worked reasoning for qualifiers and intermediate steps; the abbreviated labels do not replace that reasoning.

Before applying the procedure, write its units, denominator or decision boundary where relevant. A partner should be able to repeat each step using the same inputs. If a required input is absent or violates the stated contract, record that condition explicitly instead of quietly replacing it with a convenient value.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

Capacity 12, used 5; capacity 12, used 12; capacity 12, used 13. Show the intermediate steps and give the conclusion with its scope.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Supply zero instead of 7.

Prediction: Reject the input under the stated positive-integer contract.

Reason: Zero has the right type but violates the value requirement.

5. Words that unlock the idea

Word · Meaning

Word: Pure function

Meaning: A computation with no externally observable side effects

Word: Boundary case

Meaning: An input at a contract edge

Word: Expected outcome

Meaning: The result or error specified before execution

Use two terms to explain a step in this week's example.

6. Follow a worked example

Capacity 12, used 5; capacity 12, used 12; capacity 12, used 13.

Results 7 and 0; the third is invalid rather than remaining -1. Preserve the contract failure.

Compare the intermediate evidence with the final statement. Explain why each number or decision follows from the stated inputs. The conclusion belongs to this scenario and procedure; changing the inputs, evaluation population or authority can change what it establishes. Retain the raw record so a later revision can be compared honestly.

Optional local lab: predict the printed result before running this standalone Python 3 example. Change one relevant boundary input, record the outcome and state the contract or statistical assumptions. The program uses fictional values and the Python standard library; it performs no external operations.

```
def nonnegative_integer(value):  
    return type(value) is int and value >= 0  
  
def remaining(capacity, used):  
    if not nonnegative_integer(capacity) or not nonnegative_integer(used) or  
    used > capacity:  
        raise ValueError('capacity/used')  
    return capacity - used  
  
print(remaining(12, 5))
```

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Testing pure components

Test a room formatter alone. A pure function returns "Room " plus a supplied positive integer.

The facts we will use

Accepted inputs are positive integers. Expected outputs: 7→"Room 7"; 12→"Room 12".

AI ACADEMY / AI-6/7

Testing pure components

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Call with 7

The prefix and supplied numeral produce Room 7.



2. Compare exact text

The expected text includes one space before 7.



3. Call with 12

The same fixed procedure produces Room 12.

CHANGE ONE THING

Supply zero instead of 7.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Call with 7

What happens and why: The prefix and supplied numeral produce Room 7.

Step: 2. Compare exact text

What happens and why: The expected text includes one space before 7.

Step: 3. Call with 12

What happens and why: The same fixed procedure produces Room 12.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

A pure component depends on its accepted input, not a clock, network or model. Isolating it makes failures easier to locate.

What this example does not establish

These tests cover only stated cases; they do not test the full AI service or every invalid value.

Read the labelled picture: Predict the formatter outputs for 7 and 12, including spacing. Why does input 0 need rejection under this contract rather than an ordinary formatted response?

Change one thing: Supply zero instead of 7. Predict the result before checking the explanation. What remains the same?

7. Find and repair a mistake

Passing unit cases for a pure function does not establish integration permissions or model-dependent behaviour.

A useful check asks what would make the reasoning fail. Distinguish a failure to execute the declared procedure from a weakness in the procedure itself. The first may require a correction; the second requires an explicitly revised method and new evidence. Preserve unsuccessful cases so an improved result cannot hide the original problem.

Guided: restate the worked result, then name the assumption or limitation that prevents a broader conclusion.

Locate the earliest unsupported step

A report omits this qualification: Passing unit cases for a pure function does not establish integration permissions or model-dependent behaviour. Explain how omitting it could change a reader's interpretation, and write an accurate replacement.

State one repair and the checks you would rerun.

8. Practise together

Explain the weekly mechanism in your own words. Use the worked scenario to name its input, decision rule and output.

Guided: restate the worked result, then name the assumption or limitation that prevents a broader conclusion.

Explain your prediction to a partner before comparing results.

Practise with less help: Trace the worked case

Use workbook W2 for the connected example “Testing pure components”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Testing pure components. Proposed explanation: “The worked result establishes every future case”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Predict the formatter outputs for 7 and 12, including spacing. Why does input 0 need rejection under this contract rather than an ordinary formatted response?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

Capacity 12, used 5; capacity 12, used 12; capacity 12, used 13. Show the intermediate steps and give the conclusion with its scope.

```
def nonnegative_integer(value):  
    return type(value) is int and value >= 0  
  
def remaining(capacity, used):  
    if not nonnegative_integer(capacity) or not nonnegative_integer(used) or  
    used > capacity:  
        raise ValueError('capacity/used')  
    return capacity - used  
  
print(remaining(12, 5))
```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

Explain the weekly mechanism in your own words. Use the worked scenario to name its input, decision rule and output.

A report omits this qualification: Passing unit cases for a pure function does not establish integration permissions or model-dependent behaviour. Explain how omitting it could change a reader's interpretation, and write an accurate replacement.

For the worked scenario, propose one additional case that could reveal a failure, describe its expected outcome before running it, and explain what would need to remain fixed for a fair comparison. Do not reuse W4 as a new final test after receiving feedback.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled testing pure components evidence sheet to Grounded Learning Service. Show how you can explain the mechanism and terminology of testing pure components. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For the Grounded Learning Service, retain an annotated trace and the changed-case result. Name the contract or evidence that makes the deciding step valid, so another team can review it.

12. Show what you know

Complete the supported workbook tasks before attempting W4 independently. Record any hints or help used. After feedback, use the teacher's fresh retry rather than copying the earlier answer. For a project, select a relevant new case, predict the outcome from the declared rule and preserve the result alongside any revision.

Exit: propose one revealing change to the worked scenario. State the expected outcome and the evidence that would make you revise your explanation.

For the worked scenario, propose one additional case that could reveal a failure, describe its expected outcome before running it, and explain what would need to remain fixed for a fair comparison. Do not reuse W4 as a new final test after receiving feedback.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

Testing pure components — evidence trace

Input and scope · Check or calculation · Conclusion and limit

Input and scope: Capacity 12, used 5; capacity 12, used 12; capacity 12, used 13.

Check or calculation: Define $\text{remaining}(\text{capacity}, \text{used}) = \text{capacity} - \text{used}$ for integer inputs satisfying $0 \leq \text{used} \leq \text{capacity}$. Invalid types/ranges raise an explicit validation error. Test zero and equality boundaries.

Conclusion and limit: Results 7 and 0; the third is invalid rather than remaining -1. Preserve the contract failure.

The figure connects this chapter's worked input, declared procedure and result. Read the complete worked reasoning for qualifiers and intermediate steps; the abbreviated labels do not replace that reasoning.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Apply a new case independently

Capacity 9 with used 0, 9, 10 and Boolean True. Give outcomes.

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Capacity 9 with used 0, 9, 10 and Boolean True. Give outcomes.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.

15. Your lesson route

Start with this question: Test a room formatter alone. A pure function returns "Room " plus a supplied positive integer.

By the end, I can · How I will show it

By the end, I can: Explain the mechanism and terminology of testing pure components

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Apply the stated procedure to a new case with a traceable result

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Identify an assumption or limitation and design a revealing follow-up

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project evidence. Your teacher may spread this lesson across several classes.

Keep your first prediction. Compare it with what the evidence shows and explain any change.

16. Finish and return

Close your notes. Explain this goal in your own words: Explain the mechanism and terminology of testing pure components

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an input or assumption and justify your prediction.

Save your workbook evidence. At the next review, try a different case before opening your old answer.

17. Supported practice, repair and portfolio

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Test a room formatter alone. A pure function returns "Room " plus a supplied positive integer.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? The same fixed procedure produces Room 12.

1. Call with 12
2. Compare exact text
3. Call with 7

My selected conclusion, deciding fact and explanation

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

After your attempt, compare with the supported explanation: Call with 12 The same fixed procedure produces Room 12. A pure component depends on its accepted input, not a clock, network or model. Isolating it makes failures easier to locate.

Practice 2: explain the deciding evidence

Changed-case practice: Supply zero instead of 7. Which conclusion is supported by the supplied facts?

1. This one example guarantees correct results for every future input.
2. Reject the input under the stated positive-integer contract.
3. We can decide the result without examining the changed input or the procedure.

My selected conclusion, deciding fact and explanation

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

After your attempt, compare with the supported explanation: Reject the input under the stated positive-integer contract. Zero has the right type but violates the value requirement.

Repair a missing building block

Review Least privilege and tool scopes (ai-6/6). Goal: Explain the mechanism and terminology of least privilege and tool scopes. Short practice: Classify read S2, write S2 and read S5 as permitted or denied. Identify the operation or station mismatch for each denial, then reconsider after read S5 is added to trusted scope.. Return to this week and explain what you changed.

Review Configuration and secrets (ai-6/4). Goal: Explain the mechanism and terminology of configuration and secrets. Short practice: Point to the public setting and the protected credential. What changes when `maximum_words` becomes 60, and which information must never appear in the startup log?. Return to this week and explain what you changed.

Review Requirements and service boundaries (ai-6/1). Goal: Explain the mechanism and terminology of requirements and service boundaries. Short practice: Split the museum visitor's request into two operations. Which can the service perform for P, and what happens to each operation when Q has no approved card?. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual output or error, change one input, then run again. Keep code, predictions, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run evidence. Open the HTML copy to read or print it. Drafts stay in this browser when storage is available; clear drafts on a shared device.